CARMIX: An Operating System Organized Around Content-Addressed Rematerialization of Live Execution

Interim / Preliminary Report, pending the full technical report

Loucas Louka

2026

Abstract

CARMIX is a research operating system kernel organized around a single principle. A context switch, a page fault, a cross-machine migration, a user and kernel crossing, a program load, and a process deschedule are treated as one operation, the rematerialization of content-addressed state under a freshly minted capability whose authority cannot exceed the source authority, over BLAKE3 hashes. The contribution is this principle and its tested demonstration, not feature completeness. The system boots on x86-64 through a standard bootloader, runs a capability-guarded WebAssembly executor inside the kernel, rematerializes computation state on one machine and across two machines, persists a content-addressed object and a whole dematerialized process to disk so they survive a genuine cold reboot, reclaims unreachable objects with a durable reference-counted collector, partitions deduplication by authority domain to close the cross-domain timing channel, brings up a second CPU as a multi-core foundation, and ships a machine-checked Coq proof of its core authority guarantee. The proof is axiom-free except for one stated collision-resistance hypothesis. To keep the claims separable, every result is placed in one of three maturity tiers. Tier 1 is built in CARMIX, C-tested, and rdtsc-measured. Tier 2 is paper-argued theory not yet validated on CARMIX. Tier 3 is a research verdict or formal result whose implementation is not built. The Deterministically Reproducible Consistent Cut result is now machine-checked in the minimal model of `proofs/CarmixDrcc.v` (Qed under `coqc` 8.20.1, under named model-to-machine assumptions) and first-exercise-measured on two real cores (U-3, CV8) over the exercised workloads and schedules only; it stays Tier 2 on CARMIX because the multi-core consistent capture is not yet demonstrated in running code. The deduplication impossibility bound, the content-addressed convergence verdict, and the userland phasing are Tier 3. Every runtime number reported here was observed in the QEMU emulator and is quoted from the captured serial transcript. CARMIX is roughly forty to forty-five percent of the way to a general-purpose operating system. It is a research prototype, not a finished operating system, and the Limitations section states in full what is not built. The honesty of that list, and of the tier labels, is what is meant to make the machine-checked and tested parts credible.

1 Introduction

Operating system design has periodically been reorganized around a single unifying abstraction. Unix made everything a file, so that a device, a pipe, and a socket are reached through one descriptor interface [1]. Plan 9 carried that further and made everything a name in a per-process namespace, so that resources on remote machines are mounted and used as local names [2]. Each step took a heterogeneous set of operations and expressed them through one interface, and the gain was conceptual economy, fewer mechanisms to build, to reason about, and to secure.

CARMIX proposes the next axis. Everything is a content-addressed object, named by the BLAKE3 hash of its content, and the operations a kernel performs on live execution state are unified as one operation over those objects: rematerialization. To rematerialize a computation is to reconstruct it elsewhere from its content address rather than to copy its bytes wholesale. A context switch, a demand-paging fault, a migration to another machine, a syscall crossing, a program load, and a deschedule then differ only in scope and in where the destination state already resides. Each is a verified materialize-by-hash under a re-minted capability whose authority is bounded by the source.

The synthesis is the contribution. Content-addressed migration of program state exists in prior work, and capability systems exist in both hardware and soft-

ware forms. The new part is fusing them so that diff-proportional transfer and monotonic capability re-mint emerge together from executing live state inside a content-addressed immutable memory model, with the authority guarantee enforced on every re-mint and tied to a machine-checked proof. This report states the principle, walks the architecture subsystem by subsystem with each part tied to the milestone that demonstrates it, states exactly what the Coq development proves and its one assumption, and reports the measured numbers from the emulator with honest framing of what each shows.

This report stratifies its claims by maturity, so that no result reads as more finished than it is. **Tier 1** is built in CARMIX, exercised by its self-tests, committed to the public repository at `github.com/dlukel/CARMIX`, and rdtsc-measured in the emulator. The Architecture and Evaluation sections are Tier 1 unless a sentence states otherwise. **Tier 2** is paper-argued theory that CARMIX does not yet validate in running code. **Tier 3** is a research verdict or formal result, including clean impossibility and undecidability findings, whose implementation is not built or is only partially started. The Tier 2 and Tier 3 results have their own sections after the Evaluation, and each result is labeled with its tier where it appears.

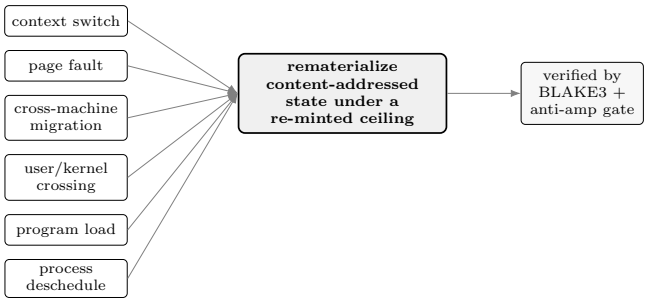

Figure 1: The one-operation funnel. Six kernel operations on live execution state are expressed as one operation, rematerialization of content-addressed state under a re-minted capability ceiling, with integrity by BLAKE3 and authority by the anti-amplification gate.

Stated operationally, when a computation's state is a graph of content-addressed immutable objects, moving it to a destination that already holds part of that graph moves only the objects the destination lacks, and the authority granted at the destination is minted fresh under a gate that cannot exceed the source authority. The first property is the structural diff. The second is the anti-amplification re-mint. Figure 1 shows the operations that CARMIX expresses through this one path.

2 The Principle

Three pieces combine into one operation.

Content-addressed state. Every object in the computation's memory graph is named by the BLAKE3 hash of its content, including the names of its children. Identical content has one name and one stored copy. Changing an object yields a new object with a new name, and the unchanged objects keep their names. There are no paths, no keys, and no overwrite.

Diff-proportional transfer. Because a destination that holds an object holds its name, a transfer sends only the objects whose names the destination lacks. Unchanged sub-objects keep their names across an epoch and are never resent, so the bytes on the wire track the size of the change, not the size of the state. This is a property of the naming scheme, not a separate deduplication pass.

Monotonic capability re-mint. A capability is a token of authority over a region of memory with a permission set. CARMIX never moves a live capability across a boundary. It strips the capability to a plain position-independent descriptor, transfers the content-addressed state, and mints a fresh local capability at the destination through a re-mint gate. The gate is fail-closed. The minted capability's bounds are no wider and its permissions no greater than the source held. Over-authority is rejected, not silently clamped.

3 Architecture

This section walks the system from the proven modules up through the kernel, and ties each subsystem to the milestone in which it was observed. Milestone codes (for example E2, R3, C3, M4) match the kernel self-test stages and the captured serial transcript. Everything in this section is Tier 1, built in CARMIX and observed in the captured transcript, with the measured costs reported in the Evaluation.

3.1 Proven modules

Five modules below the kernel hold the model and ship byte-identical to their verified form.

Capability model (cap/). A capability is a record of a base, a length, a permission set, and a validity tag. *Strip* turns a live capability and a reference to its referent into a plain descriptor carrying offset, length, permissions, and the content hash of the referent, with no live authority tag. *Re-mint* takes a destination root, a descriptor, and the destination's copy of the referent, and mints a fresh capability, checking in order that the descriptor is well formed, that its bounds fit the source region, that its permissions are a subset of the source, that write and execute are not both requested, and that no forbidden permission is present. Any failure returns an invalid capability.

Content-addressed object store (store/). The store maps a BLAKE3 hash to bytes. Putting bytes returns their hash and stores one copy. Putting identical bytes again stores nothing new. The vendored BLAKE3 is validated against the official known-answer vectors, and the kernel re-runs that known-answer check at startup before it trusts any content-addressed result (milestone R0, digest of the empty string matches `af1349b9..41f3262`).

Single-level store and structural diff (sls/). The single-level store builds an object graph over the store. A leaf holds bytes. A node holds an ordered list of child identities. Mutation produces a new object and a new epoch, and unchanged sub-objects keep their identities. The structural diff takes two epoch roots and returns the changed set, pruning any subtree whose hash is unchanged, memoizing visited identity pairs, calling no hash function, and resolving only as many objects as actually changed.

WebAssembly safety gate (gate/). The gate runs untrusted WebAssembly under software fault isolation [3, 4]. It validates a WebAssembly subset, lowers it to a small typed intermediate representation, and runs a linear-time checker that rejects any module that could escape its linear memory or forge a capability. A static elision pass drops a runtime bounds check only for an access it has proven in bounds, and the checker rejects the elided opcodes in untrusted input so they cannot be injected.

Software capability backend (carmix/). The backend enforces the capability model with no capability hardware. Its re-mint logic is the same decision logic as cap/, ported to software records with integer comparisons and no rounding. This backend is what the bootable kernel uses, and the binary contains no capability instructions.

3.2 Boot and in-kernel execution

The kernel boots on x86-64 through the Limine boot protocol, which hands it long mode, page tables, a memory map, and a linear framebuffer. Everything above the entry point is written for CARMIX. The boot milestones B0 through B5 confirm serial output, long mode (`CS=0x28`, `EFER.LMA=1`), a physical frame allocator with read-back, a $1280 \times 800 \times 32$ framebuffer demonstrated by pixel read-back, and a PIT timer. The gate and the software backend, linked into the kernel, run a WebAssembly task and draw its result (E1, result = 51), and the anti-amplification property holds in the kernel with one control accept and six attack classes each rejected by its intended check (E2, 6/6). The store and the single-level store, linked in, checkpoint a counter and rematerialize it bit-identically (R3), and demonstrate the diff-proportional structural diff (R4, a one-leaf change reports one changed object, a self-diff reports zero, a three-leaf change reports three).

3.3 Desktop

A software compositor draws a background and windows back to front with a save-under cursor, driven by polled PS/2 keyboard and mouse. Windows support focus raise on click, drag by the titlebar, and resize by a corner grip, each demonstrated by framebuffer pixel read-back. In the headless self-test run these events are scripted and labelled as such. Only the final shell stage takes live PS/2 input. The desktop draws CARMIX's own windows in software and does not run third-party graphical applications.

3.4 Two-machine migration and signed authorization

Two emulator instances share memory through an ivshmem device used as a byte channel. The migration is a destination-pull protocol: the source advertises its epoch root, the destination walks the root top down and marks the hashes it lacks, requests them, and verifies each block's BLAKE3 hash before installing it, pruning any subtree it already holds. A block whose recomputed hash does not match the requested hash is rejected fail-closed.

Hash verification proves integrity, not provenance. A signed authorization record closes that gap. The record binds the migration to an epoch root, a computation identifier, a destination identifier, an authority ceiling, a nonce, and an expiry, and the source signs it with real Ed25519 (vendored tweetnacl [5, 6]). Before it installs and re-mints, the destination checks, each fail-closed with a distinct reason, that the signature is valid under the expected source key, that the signed root equals the root it pulled, that the destination and computation identifiers match, that the nonce is unseen and unexpired, and finally that the re-mint is capped at the authority ceiling through the anti-amplification gate. One legitimate migration is accepted and seven adversaries are each rejected by their own reason (milestone D2).

3.5 Key lifecycle

The destination holds a trust store of an authority public key, the current source public key, a monotonic lifecycle epoch, and a permanent revoked set. Authority-signed enrollment sets the first source key, rotation replaces it, and revocation adds a key to the revoked set. A lifecycle record applies only if it verifies under the authority key and its epoch strictly exceeds the stored epoch, which rejects replay and downgrade. Each migration carries its signing key, which must be the current source key and not revoked. Six lifecycle attacks are each rejected by a distinct verdict (milestone KA).

The destination's first authority key is baked in. Its bootstrap is out of scope and stated as such.

3.6 The task substrate

The kernel creates tasks and switches between them with a real assembly context switch that saves and restores callee-saved registers and the stack pointer. A cooperative yield calls it directly, and the timer interrupt calls it too, so a task that never yields is still preempted (milestones P0 through P2). The scheduling policy is isolated in one function, `pick_next`, which is round-robin and is a deliberate placeholder.

3.7 The unified content-addressed task substrate

On top of the substrate a task is made a content-addressed object. Its registers and the used part of its stack hash into the content-addressed store, the same state hashing to the same handle and different state to a different handle (U0). Activate-by-hash dematerializes the task to a hash and materializes it back, and the counter survives the round trip (U1). Persistence falls out of this: a task dropped and resumed from only its hash and the store resumes correctly, so the live and durable forms are one object (U4). The cost is measured, not assumed. A raw register switch is a flat cost near 1062 cycles, while activate-by-hash costs two to three orders of magnitude more at every dirty-set size, because hashing dominates, with no dirty size at which it beats the register switch (U3). So the fast switch stays the path for rapid preemption and content-addressing is used only at coarse boundaries that already touch the store. The page-table root and the capability slots are not yet part of the task state object.

3.8 The residency manager

Physical memory is managed as a content-addressed cache over the store. Fresh allocation and mapping stay conventional. Fault-in and sharing collapse into rematerialization. Free, eviction, and defragmentation partially collapse. `materialize(hash)` fills a frame and verifies that its BLAKE3 equals the requested hash, so integrity holds by construction (M2), a request for a resident hash shares the existing frame and bumps its refcount, eviction writes a dirty victim back before reuse with temporal victim selection (M3), and the hardware page-table dirty bit is measured against the software chunk diff: both find the same dirty set $\{1, 4, 6\}$ of eight pages and the hardware scan is roughly nine times cheaper (420,310 against 3,956,943 cycles, M4). Fixed chunks trade external fragmentation for internal slack near 29 percent on the measured mix, and a frame-sized request stays serviceable (M5).

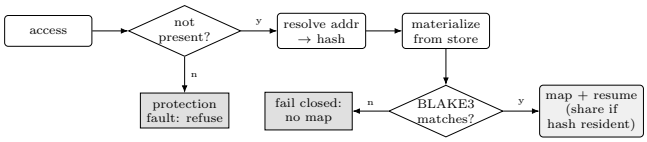

Figure 2: The residency and fault path (F1, F2, F3). A miss is a verified materialize-by-hash. A protection fault and a verification failure are each refused at the fault boundary, and an already-resident hash is shared.

3.9 The rematerializing fault handler

Figure 2 shows the residency and fault path: a not-present access is serviced as a verified materialize-by-hash, and a verification failure or a protection fault is refused at the fault boundary.

Demand paging is automatic. A not-present access traps through the page-fault vector, the handler classifies the fault (a protection fault is not a miss), resolves the faulting address to a content hash, materializes the object with BLAKE3 verification, installs the mapping, and returns so the faulting instruction re-executes (milestone F1, a live page fault). If the loaded bytes do not hash to the expected hash, the handler installs no mapping and does not resume (F2). Two addresses bound to one hash share one resident frame, so deduplication happens on the fault path (F3). Two address-to-hash bindings were measured head to head and the choice does not move fault-service latency, because the cost is dominated by the materialize and the hash (F4, a null finding). The handler's own memory is resident so servicing a miss does not itself fault. A fault during fault handling fails closed rather than recovering, and making it recoverable needs a separate interrupt stack and a per-fault service context, named as future work (F5).

3.10 Authority-bounded user and kernel crossing

The kernel runs a process in ring 3 on its own page tables, with a task-state segment, a trap gate, and the user and supervisor bit on the page tables. A ring-3 access to a kernel address faults and is classified as a protection violation, not a miss (US0). Entering the kernel is not ambient privilege: a privileged crossing re-mints the caller's bounded authority for the requested service through the same anti-amplification gate that bounds a migration and a fault, so a confused deputy that hands the kernel a wider claim is held to the re-minted authority (US2). Six userspace amplification attempts, an unheld capability, an over-ceiling request, a forged epoch, a confused deputy, a write-execute violation, and a forbidden permission, are each refused by their own distinct status code, the same codes the migration table uses, with a replay refused and a legitimate crossing

accepted (US3). A large syscall argument is passed as a hash and rematerialized by hash with verification rather than copied, which closes the copy-at-use class of time-of-check to time-of-use race, since a mutated object is a different hash and fails verification. This relocates trust to the address-to-hash binding and the store admission rather than eliminating it (US4).

3.11 The process loader

A program is a content-addressed object and loading a process is materialization. A freestanding ring-3 ELF64 is stored, the loader parses its program headers, and for each loadable segment it materializes the bytes by hash with BLAKE3 verification into a fresh ring-3 address space, with the segment's permissions and no segment both writable and executable (L1, W⊕X held). The process runs under a capability re-minted through the gate at load time, so its authority is bounded at birth and an over-ceiling or unauthorized request is refused by the same gate (L3, each refused by a distinct reason). Loading the same image twice gives two processes whose read-only code resolves to one physical frame by hash while their writable data and stacks are private (L4, code refcount = 3). The image is embedded in the kernel because there is no filesystem yet.

3.12 Concurrency

The scheduler runs more than one ring-3 process at once, each with its own page-table root, instruction and stack pointers, and re-minted ceiling, and the timer preempts and restores exactly (C0). Each process is bounded by its own ceiling and neither can exercise the other's authority (C1). A descheduled process is dematerialized to a hash and its frames released, so not running and not resident become different states, and on reselection the state is rematerialized with verification and the process resumes, its counter surviving (C2). The cost is measured: resuming a resident process is a register and page-table-root restore near 13,958 cycles, and resuming a dematerialized process is a store fetch, a verification, and a remap near 1,029,941 cycles, about seventy times more on this run, with the run-to-run variation expected under emulation (C3, see the Evaluation). Today the register context and the private data page dematerialize while the page-table root stays resident.

3.13 The policy, fairness, and the heap

A rematerialization-aware policy decides when to dematerialize. By default a descheduled process is kept resident, since resuming it is far cheaper. Dematerialization happens only under memory pressure when the process is predicted to stay descheduled long enough to repay the rematerialize cost, with a break-even duration computed live from the measured resume cost rather than a guessed constant, and an anti-thrash backoff that keeps a just-rematerialized process resident (milestones PP0 through PP4). A fairness control sits on top: a process repeatedly paying the rematerialize cost accumulates a measured per-process progress deficit, and a bounded keep-resident bias removes the penalty source so its progress rate recovers without starving its resident peer (FA0 through FA2). The control bounds future unfairness and recovers the rate. It does not refund the penalty already paid and accounts for one peer rather than many, so it is a first measured control on a lightly-researched dimension, not a proven-optimal algorithm. A per-process heap grows through an authority-bounded extend-heap that crosses the re-mint gate, backed demand-zero (PM0, PM1). An idle page can be dematerialized to a hash and rematerialized on fault (PM2), two processes that independently write the same bytes deduplicate to one frame by hash with copy-on-write where copy-on-write structurally cannot, since there is no shared ancestor (PM3), and a hot churning page is excluded from dematerialization because re-hashing it on every attempt is the fine-granularity cost the residency measurement showed (PM4).

3.14 Durable content-addressed persistence

The store is made durable on a virtio-blk disk, so a content-addressed object and a whole dematerialized process survive a genuine cold reboot (milestones DUR S1 through S7, commit `faebbd6`). A minimal polled virtio-blk driver provides stable-media block read, write, and flush (S1). Objects go to an append-only log, never overwritten, each as a header block carrying the claimed length and claimed BLAKE3 written before its payload block, so an ordered-prefix truncation leaves a partial record whose claim is present but whose payload re-hashes to a mismatch and is rejected (S2). The hash-to-location index is not itself persisted. It is rebuilt on every boot purely by scanning the log and re-verifying each record, which is the recovery floor and also the evidence that RAM started empty, since the index can only come from disk (S3).

The disk is treated as untrusted media and re-verified on every load, and the cold reboot is genuine, each boot a separate QEMU process on the same host disk image with empty RAM. An object written and flushed, then revived after a cold reboot, rematerializes by its BLAKE3 hash with the on-disk bytes re-hashed before they are served (S5). The headline is the persisted process (S6). A process is dematerialized to its page-table-inclusive canonical task hash, its object graph and a root block are persisted with the authority ceiling inside

the hashed manifest, the machine is cold-rebooted, the store is revived from disk with every object re-hashed on load, and the process is rematerialized with its authority re-minted through the same anti-amplification gate at the persisted ceiling. The address-space root re-derived from the disk-loaded state equals the persisted root, so the page-table-inclusive state is intact, and a load that requests authority wider than the persisted ceiling is refused by the gate with its distinct amplification status. Because the ceiling lives inside the hashed manifest, tampering it changes the task hash and is caught by re-verification. Tampering is caught on both an in-kernel manifest flip and a host-side payload flip between boots, each rejected on the recovery scan or the load with the bytes never served (S7). The crash consistency here is recovery-logic consistency under an ordered-prefix write-order model, exercised exhaustively and reported with that scope in the Evaluation and the Limitations. The honest scope is a single durable writer and objects up to one payload block.

3.15 Durable garbage collection

The durable log would otherwise grow without bound, so unreachable objects are reclaimed by a durable reference-counted collector (milestones GC1 through GC7, commit `40c99cd`). No cycle collector is needed, because the content-addressed graph is acyclic by construction. An object's BLAKE3 is computed over its content including its referents' hashes, so a referent's hash must exist before its referrer's hash can be computed, and an object cannot name its own or an ancestor's hash (GC2). This is the runtime face of the Coq `acyclic_graph` result. Reference counts are mutable metadata about immutable objects, so they live outside the hashed content, recorded as durable count-delta records in the same append-only log under the same ordered-prefix discipline, and on a genuine cold reboot the counts are reconstructed exactly by replaying those records (GC1). The root set is the union of durable roots, the persisted root block and object index, and volatile roots held in live process state (GC3).

Reclamation is crash-consistent with the tombstone as the commit point (GC4). The reference removal and its count decrement are recorded first, the tombstone is written second, and the storage is freed last. Truncating the decrement-then-tombstone sequence at every byte offset, the forbidden state of a tombstone present while a durable reference still names the object never occurs, so a durable tombstone implies the count reached zero and no durable root reaches a tombstoned object. Because deduplication is already present, a dedup hit can arrive in the window between zero-count detection and the tombstone commit. That window resurrects the zombie object, restoring its count with the same storage block intact rather than freeing and recreating it (GC5). End to end, dropping a reference reclaims the unreachable subgraph while reachable objects stay intact and load-verify, and re-allocation reuses the freed blocks so the high-water mark does not climb (GC6). The crash-consistency scope is the same ordered-prefix-in-QEMU scope as the persistence proof. Concurrent tracing against a live multi-threaded mutator and any cross-core reclamation race are not implemented and wait on the SMP work.

3.16 Domain-partitioned deduplication

Deduplication is scoped to an authority domain, so an observer in one domain cannot probe, by store hit-or-miss timing, for content held by another domain (milestones DS1 through DS6, commit `4dd97a5`). The reference-count table key gains an authority-domain tag, extending the collector's table rather than forking a parallel one. Identical content stored in two domains produces two distinct physical objects with independent counts (DS1), and a store deduplicates only against existing objects in its own domain, a same-domain re-store hitting and a cross-domain store missing (DS2). The security result is the closed channel, stated precisely. The cross-domain store-hit-or-miss timing channel is closed by partitioning, measured in the Evaluation, where a cross-domain probe now costs what a genuinely new store costs rather than the fast hit it cost under global deduplication (DS3). The resurrection window is made domain-aware, so a zombie in one domain is not resurrected by a same-content store in another (DS4), and reclamation still works under the domain-tagged counts (DS6). This is not a general zero-leakage result. The within-domain timing channel remains, and it is acceptable because the observer is already in that domain. The cost is duplicate storage for content shared across domains, measured (DS5). The precise claim is cross-domain channel closure at a measured storage cost. The formal boundary this responds to is a Tier 3 finding stated later.

3.17 The versioned capability gate

A single-CPU versioned (MVCC) capability slot gives an authority transition a single atomic commit point, so a capture firing at any instant reads a committed pre-state or post-state capability and never a torn one (milestones VG1 through VG5, commit `faae1d6`). The slot is a version word plus two capability buffers, the active buffer selected by the low bit of the version. A replacement is written to the inactive buffer, and the commit is one naturally-aligned eight-byte store to the version word, which x86-64 makes atomic (VG1). The anti-amplification check is the existing gate, reused unchanged, so the committed capability is exactly what the C-tested re-mint check produces and never wider,

and a request for wider bounds or an added permission is refused with its distinct status while the active capability is left unchanged (VG2). Driven through sixty-four surrender-and-replace operations with captures fired at many points across each operation, including during the inactive-buffer write, every capture read a complete well-formed capability and none was torn, while a naive single-buffer slot captured mid-write reads a torn capability that matches neither the pre-state nor the post-state (VG3). The in-flight authority transient that previously needed a drain collapses to the commit bump. This is demonstrated on a single CPU, where there are no remote cores and no stale TLBs. It does not implement or validate the multi-core memory cut or the cross-core capture, which wait on the SMP work and on the Tier 2 result below. Its overhead is reported in the Evaluation as small and at or below the measurement noise of the re-mint it wraps, not a speedup.

3.18 SMP bring-up: the multi-core foundation

A second CPU is brought up, giving CARMIX a genuine multi-core foundation (milestones SMP S1 through S4, commit `40c99cd`). The run is under QEMU with multi-threaded TCG, so the two virtual CPUs execute on separate host threads and are genuinely parallel. Limine performs the standard INIT-SIPI-SIPI and hands each application processor to the kernel in long mode, and the second core reaches kernel C code and announces its LAPIC id on serial, which is observable evidence a second core is executing (S1). Each CPU has its own per-CPU record and increments only its own counter (S2). The Local APIC is initialized in xAPIC mode on each core, and a real inter-processor interrupt is delivered and handled in both directions with an end-of-interrupt acknowledgement, which is the primitive every future cross-core operation will build on, including the TLB shootdown a multi-core capture would need (S3). The cross-core sanity test runs the same work on both cores concurrently. Under a spinlock the shared counter totals exactly its expected value, so the lock holds across real cores, and without the lock the counter loses updates, which can only happen under truly concurrent read-modify-write and is therefore positive evidence of parallel execution (S4).

This is the foundation only. It is engineering, not a research result, and it does not by itself validate the multi-core consistent capture or the Deterministically Reproducible Consistent Cut. After the test the application processor parks and the single-CPU path runs as before. What this foundation unlocks, and the gap that remains, is the subject of the Tier 2 section.

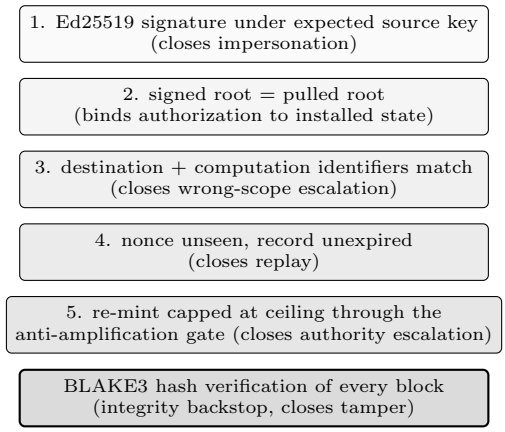

Figure 3: The trust-model layers at the migration boundary. Each check is fail-closed with a distinct reason, applied before any state is installed or any capability is re-minted. The key-lifecycle current-key and not-revoked checks run ahead of layer 1.

4 Trust Model and the Anti-Amplification Proof

4.1 Threat model

The two-machine threat model covers tamper (a block corrupted in transit), impersonation (a party that is not the legitimate source induces a rematerialization), replay (an old signed migration presented again to force a stale epoch), and scope escalation (state for a different computation, destination, or higher ceiling). The signing key itself is a target, with unknown key, rotation forgery, rotation replay or downgrade, use after revoke, and revocation forgery. Hash verification closes tamper. The signed authorization record closes impersonation and scope, and the key lifecycle closes the key attacks, each fail-closed with a distinct reason as Section 3 and the Evaluation report. Figure 3 shows the layered checks.

4.2 The anti-amplification property

The anti-amplification property is that a re-minted capability's base is no lower and its limit no higher than the source region, and its permissions are a subset of the source permissions. Figure 4 shows the gate. It is what makes crossing, migrating, faulting, loading, and descheduling one authority model: each goes through the same re-mint, and the gate is fail-closed, so over-authority is rejected rather than clamped.

4.3 What the Coq development proves

The proof in `proofs/Carmix.v` models a capability as a record of a base, a limit, a finite permission set, and

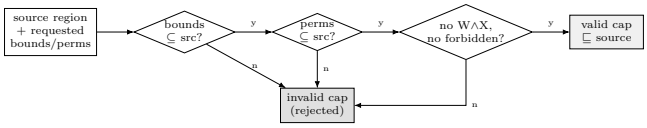

Figure 4: The anti-amplification re-mint gate. Every authority crossing in CARMIX, migration, fault, user/kernel crossing, load, and deschedule, passes through this one fail-closed gate.

a validity flag, with a forbidden permission subset. It defines an authority order in which one capability is below another when its validity implies the other's, its range sits inside the other's, and its permissions are a subset. It defines a re-mint function that clamps the requested range to the source, intersects the requested permissions with the source, rejects a request for write and execute together or for any forbidden permission, and sets validity accordingly. Three results are machine-checked, labeled C1, C2, and C3.

- C1, `anti_amplification`: for all sources and requests, if the re-minted capability is valid then it is below the source in the authority order.
- C2, `valid_dest_no_wx` and `valid_dest_no_forbidden`: a valid re-minted capability does not hold write and execute together and holds no forbidden permission.
- C3, `acyclic_graph`: the content-addressed object graph is acyclic.

C3 is what makes the durable reference-counted collector of the Architecture sound. An acyclic graph cannot form a reference cycle, so refcounts reach zero exactly when objects become unreachable and no cycle collector is needed.

The development is axiom-free except for one stated hypothesis. The acyclicity result is scoped honestly: it does not derive acyclicity from hashing alone. Objects are modeled as a finite inductive tree, a node carrying a payload and a list of child objects, so a child is a strict subterm of its parent, a structural size measure strictly decreases along the child relation, and acyclicity follows from that well-founded order. The single hypothesis, that the identity function is injective (collision resistance), is what connects the hash-graph back to that finite tree. The size measure is a proven lemma, not an assumption. The development contains no `Admitted`, no `admit`, and no `Axiom`. `coqc` and `coqchk` both accept it, and an audit reports the three authority results as closed under the global context and `acyclic_graph` as carrying only the collision-resistance hypothesis.

A separate file, `proofs/CarmixDag.v`, strengthens the acyclicity result from that finite inductive tree to a genuinely shared, hash-consed DAG, the store shape a content-addressed system actually has, where one child hash sits under several parents as a single resident object. It proves that store graph acyclic under two named assumptions of the file, `H_CF` (store-level collision-freedom) and `H_WF` (a hash-before-name creation order), with every proof closed by Qed under `coqc` 8.20.1, and it still does not derive acyclicity from hash collision-freedom alone.

The proof is about the abstract capability algebra. It is not a proof that the C implementation in `carmix/swcap.c` refines this model. The runtime correspondence is demonstrated, not proven, by the kernel attack tables (E2, US3, D2, KA), each of which runs the real software backend and observes each adversary rejected by its intended check. A machine-checked refinement of the C code against this model is future work. One rung of it is now decided rather than tested: `proofs/cbmc` records a bounded, bit-precise CBMC decision of the real gate `cvsasx_swcap_check`, which within stated bounds decides that the gate grants if and only if the access is valid, the requested permissions are a subset of the capability's (no amplification), and the requested window lies within the capability's length. `cbmc` 6.6.0 reports VERIFICATION SUCCESSFUL, 0 of 27 properties failed; the reachable code has no loops or recursion, so the decision is exhaustive over the nondet input domain. This is a bounded decision within stated bounds, not a Coq Qed proof.

5 Evaluation

All runtime numbers below were observed in the QEMU emulator and are quoted from the captured serial transcript. Cycle counts are `rdtsc` differences taken under TCG emulation, so they are noisy run to run. That variation is real and is reported as such, and the point of each measurement is the order of magnitude and the direction, not a precise constant.

5.1 The cost crossover

A raw register context switch is a flat cost near 1062 cycles, independent of dirty-set size (U3). Activate-by-hash, which serializes and hashes the task state, is two to three orders of magnitude more at every swept dirty size, and there is no dirty size at which it beats the register switch, because the BLAKE3 hash of even one chunk already dwarfs the register swap. This is the load-bearing negative result: a scheduling policy that content-addresses on every switch is not viable on this hardware, so content-addressing is reserved for coarse, infrequent boundaries that already touch the store. Table 1 reports the incremental-cost measurement (U2) that anchors it: re-storing only the dirty set tracks the dirty set, with a one-chunk change far cheaper than a

Table 1: Incremental dematerialization tracks the dirty set (U2). A 4096-byte page in 16 chunks, measured cycles are `rdtsc` under TCG.

case	dirty chunks	bytes	cycles
baseline (all dirty)	16	4096	1,066,976
small (1 chunk)	1	256	278,448
large (all chunks)	16	4096	815,649

Table 2: Net outcome of dematerializing across deschedule duration, anchored in the measured C3 resume cost (PP3). Net is freed-frame value over the deschedule minus extra resume cost. Break-even $D^* = 493{,}223$ cycles.

duration (cyc)	freed value	extra cost	verdict
123,305	253,993	1,015,983	loss
246,611	507,989	1,015,983	loss
493,223	1,015,981	1,015,983	loss
986,446	2,031,962	1,015,983	gain
1,972,892	4,063,924	1,015,983	gain

full re-store.

5.2 Resume cost and the break-even

The deschedule resume path was measured directly (C3). Resuming a resident process, a register and page-table-root restore, cost 13,958 cycles. Resuming a dematerialized process, a store fetch, a BLAKE3 verification, a writeback, and a remap, cost 1,029,941 cycles, a ratio near 73× on this run. The architecture narrative describes this path as about fifty times more expensive. The spread between that figure and the 73× measured here is the run-to-run emulation noise the transcript notes explicitly. This single measured cost anchors the policy. The break-even deschedule duration, the point past which freeing a frame for the deschedule duration outweighs paying the extra resume cost, is computed live from it at 493,223 cycles (PP3). Table 2 reports the net outcome across deschedule durations: shorter than the break-even nets a loss, longer nets a gain. The anti-thrash backoff bounded a forced thrash workload to a single dematerialization across seven rounds, keeping the process resident on the other six (PP4).

5.3 The dirty-bit comparison

The unified-substrate crossover left open whether a cheap coarse filter could front the content-addressing. The residency manager answers it (M4). A workload wrote pages $\{1, 4, 6\}$ of eight. The hardware page-table dirty-bit scan and the software `memcmp` chunk diff agree on the dirty set, and the hardware scan cost 420,310 cycles against the software diff's 3,956,943, roughly nine times cheaper. The hardware bit is the cheap coarse filter and content-addressing is the fine mechanism, and they compose.

Table 3: Two-machine diff-proportional migration, bytes on the wire, measured between two emulator instances (milestones B2, B4).

sync	blocks	bytes received	store objects
cold	1093	44,505	1093
warm hop 1	4	1176	1097
warm hop 2	4	1176	1101
warm hop 3	7	2219	1108

Table 4: Runtime adversarial tables. Each boundary runs the real software backend and observes each adversary rejected by its intended check, with a control accept.

boundary	milestone	accept / reject
in-kernel anti-amp	E2	1 / 6
user/kernel crossing	US3	1 / 6 (+ replay)
two-machine migration	D2	1 / 7
key lifecycle	KA	legit flow / 6

5.4 Two-machine migration

Table 3 reports the bytes on the wire between two emulator instances. The cold sync transferred the full graph, 1093 objects in 44,505 bytes. Three warm hops after small changes transferred 1176, 1176, and 2219 bytes. The third hop changed two leaves and fetched more than the first, which changed one, so the warm transfer tracks each hop's own change set rather than a constant. No block was rejected on these clean runs. A corrupted or forged block is rejected fail-closed by the BLAKE3 hash check.

5.5 The adversarial tables

The anti-amplification gate was run against attack classes at each boundary, and each adversary was rejected by its own intended check, with a control accept. Table 4 summarizes the runtime tables. The in-kernel anti-amp run rejected 6/6 (E2). The user and kernel boundary rejected six amplifications by six distinct status codes plus a replay, with a legitimate crossing accepted (US3). The migration trust boundary accepted one legitimate migration and rejected seven adversaries each by a distinct reason (D2). The key lifecycle accepted the legitimate enroll, rotate, and revoke flow and rejected six lifecycle attacks each by a distinct verdict (KA). The status codes are shared across the boundaries, which is the evidence that it is one gate.

5.6 Sharing and dedup

Sharing a hash at admission re-materialized zero bytes and cost 116,891 cycles, far less than a real fault at 2,456,493 cycles per fault that moved 4096 bytes, so deduplicating on the way in beats a post-hoc merge (M2). On the fault path two addresses bound to one hash shared one frame with a single store fetch (F3). Two processes that independently wrote the same heap page, with no shared ancestor, deduplicated to one physical frame by hash, and a later write split a private copy under copy-on-write (PM3). A hot churning page is excluded from dematerialization. Over a burst of 64 rewrites the exclusion avoided about 26,689,536 cycles of repeated hashing (PM4). The fairness measurement (FA0) showed a process dematerialized every turn paying 12,191,796 cycles of rematerialize penalty over twelve equal turns that its resident peer did not pay, and the keep-resident control stopped the deficit from climbing and recovered the victim's progress rate (FA1, FA2).

5.7 Durable persistence cost and the crash proof

A per-object durable write plus flush on the virtio-blk path cost 3,561,112 cycles (DUR S1). After a cold reboot, re-verifying and indexing six objects by scanning the log cost 46,884,349 cycles, and the index came purely from disk (S3). A cold-reboot object round-trip, the lookup, read, re-hash, and compare, cost 3,099,346 cycles (S5). The crash proof read the durable two-record log back into its exact byte image and ran the boot recovery parser at every one of the 16,385 byte offsets of the write, modelling an ordered-prefix truncation at each point. At all offsets the invariant held with zero violations, the recovered store holding either the complete object whose bytes hash to its name or no such hash, and never a hash naming torn bytes. The rejection branch fired at 8,896 offsets, every offset where a record's claim was present but its payload incomplete, and the same branch fires on a one-byte payload tamper. The proof pass cost 609,903,419 cycles (S4). The scope is recovery-logic crash consistency under an ordered-prefix write-order model in QEMU. Real-hardware write reordering, cache and barrier semantics, and sub-sector tearing are out of scope and named in the Limitations.

5.8 The versioned gate overhead

The versioned gate was measured against the unversioned re-mint over twenty-thousand-iteration averages (VG4). The unversioned gate operation, a re-mint into a single slot, cost 868 cycles, and the versioned operation, a re-mint plus the inactive-buffer write plus the atomic commit, cost 780 cycles on this run, so the measured versioning delta was zero this run and about +168 cycles on an earlier run. The honest finding is that the added work, a four-field buffer copy and a single word bump, is small relative to the re-mint and at or below the measurement noise on this hardware. It is not a speedup. The commit point alone, the version bump, cost 14 cycles, near the rdtsc resolution floor, and the per-slot memory cost is 32 bytes for the second buffer and the version word.

5.9 Durable GC cost and bounded growth

An in-RAM reference-count increment or decrement cost 731 cycles and the zero-count reclamation decision cost 40 cycles (GC7). The crash-proof pass over the 8,193 truncation points of the decrement-then-tombstone sequence cost about 124,490 cycles with zero invariant violations (GC4). End to end, dropping a reference reclaimed three unreachable objects and 12,288 bytes while the reachable object stayed intact, and reallocating three objects reused the freed blocks so the high-water mark held instead of climbing (GC6). The log no longer grows without bound.

5.10 Dedup domain-partitioning: the closed channel and its price

The cross-domain timing channel and its cost were measured directly (DS3, DS5). A genuinely new store, a miss, cost 1,275,657 cycles including the object write and flush, and a within-domain hit cost 2,245 cycles. A cross-domain probe, probing from one domain for content another domain holds, cost 42,873 cycles under global deduplication, a fast hit that leaks the content's existence, against 1,285,349 cycles under domain-scoped deduplication, the same order as a genuinely new store. The timing under scoping no longer distinguishes exists-in-another-domain from does-not-exist-anywhere, so the cross-domain channel is closed. The within-domain hit at 2,245 cycles remains, the acceptable residual. The price is duplicate storage. The same content across three domains stored three physical objects where global deduplication would store one, a measured duplicate-storage cost of 8,192 bytes.

5.11 SMP bring-up and the cross-core race

AP startup latency, measured from writing the application processor's entry address to the processor signalling it is up, was 599,023 cycles (SMP S1). In the cross-core test each core attempted one hundred thousand increments of a shared counter. Under the spinlock the counter totalled exactly 200,000, so the lock holds across

cores, and without the lock it reached only 112,245 of the 200,000 attempted, losing 87,755 updates, which is positive evidence the two cores ran truly in parallel (S4). No inter-processor interrupt round-trip latency is reported because it was not measured in isolation on this run. The functional result, delivery and handling in both directions, is what is reported (S3).

5.12 What the evaluation shows

The measured advantage is repeated migration or deschedule of a large state with a small change, where the transfer or the incremental store tracks the change. The crossover and the resume cost show that content-addressing is not free and must be confined to coarse boundaries, which is why the fast register switch stays the common path. The dirty-bit result shows the cheap hardware filter composes with the fine content mechanism. The adversarial tables show the one gate holds at every boundary. The persistence and GC numbers show the durable substrate is crash-consistent under the named ordered-prefix model and bounded in growth, the dedup numbers show the cross-domain timing channel closes at a measured storage cost, the versioned gate adds overhead at or below the measurement noise, and the SMP foundation runs two cores genuinely in parallel. None of this is a claim about raw compute speed, cold-start migration, or operating-system completeness.

6 Formal Results Not Yet Validated on CARMIX (Tier 2)

The result in this section is now machine-checked in a minimal model, not only paper-argued. `proofs/CarmixDrcc.v` proves, with Qed under `coqc` 8.20.1, that for a data-race-free program all schedules reaching the cut yield one canonical content name; the in-model theorem needs no Coq hypothesis given its explicit DRF premise, and the model-to-machine assumptions a deployment rests on are named as honest gaps rather than smuggled in as proved. The SMP foundation of the earlier sections has narrowed the validation gap further, but CARMIX does not yet exhibit the multi-core capture in running code, so the result is Tier 2 and not Tier 1.

6.1 Deterministically Reproducible Consistent Cut

A consistent capture of a multi-core computation needs a cut of the global state that no later replay can observe to be inconsistent. The Deterministically Reproducible Consistent Cut, DRCC, is the formalization of that property for content-addressed rematerialization. The result is a split along the data-race line. For a data-race-free program, a cut taken at release-consistency synchronization points establishes DRCC without a global stop of all cores, because the synchronization points order the memory operations that matter and the unsynchronized operations cannot be observed to differ. For state with a genuine data race, DRCC is undefined, and no capture mechanism rescues it, because the racy outcome is not a function of the cut. This is a derivation that combines the release-consistency memory model with the consistent-cut and happened-before line of distributed-snapshot work, applied to the content-addressed setting, rather than a standalone discovery.

The validation gap is explicit. The SMP bring-up of the Architecture is the foundation a multi-core capture would run on, and it now exists, so the gap has narrowed from where it stood when only a single CPU ran. But the multi-core consistent-capture cut is not exhibited or validated on CARMIX. The versioned capability gate removes the single-CPU authority transient, and the inter-processor interrupt path is the primitive a cross-core TLB shootdown would use, but neither is the cut. CARMIX does not yet demonstrate multi-core consistent capture. Doing so on the now-built SMP foundation is the first item of future work.

7 Research Findings and Verdicts (Tier 3)

The findings in this section are research results whose implementation is not built, or where only a derivation or an impossibility argument exists. A clean negative, an impossibility or an undecidability, is itself a contribution, because it bounds what any implementation could achieve. Each finding is labeled, and where a finding has a built response that response is named and placed in Tier 1.

7.1 The deduplication side-channel impossibility

Cross-process content deduplication is a known timing and occupancy side channel. A store that deduplicates globally answers a store of content another party holds faster than a store of genuinely new content, which leaks the content's existence. The formal finding, borrowed and applied from the Armknecht et al. bound, is that zero cross-domain leakage with nonzero cross-domain sharing is impossible. Any system that shares stored content across a trust boundary leaks across that boundary, so the only leak-free option for the cross-domain channel is to not share across it. This is the argued impossibility boundary. Its prescribed fix, domain-partitioned deduplication, is built and measured and

is the Tier 1 result in the Architecture, which closes the cross-domain channel at the measured cost of duplicate cross-domain storage, exactly the trade the bound makes unavoidable. The copy-on-write break channel, the other classic deduplication side channel, is structurally absent in CARMIX, because its objects are immutable and a write produces a new object with a new name rather than breaking a shared page.

7.2 Content-addressed convergence

Convergence asks whether two computations that reach the same state can be made to share one content-addressed name for it. The verdict is split, and it is not built. On the positive side, a computable canonical form exists for structural equivalence of reachable state, and it preserves both determinism and anti-amplification, provided the authority metadata is part of the hashed content so that two states of differing authority cannot collide to one name. The tractability of computing that canonical form is in principle only. It is polynomial for graphs of bounded degree, by the bounded-valence graph isomorphism result, and CARMIX's object arity is not measured, so the polynomial claim is not established for CARMIX's graphs. On the negative side, observational or behavioral equivalence is undecidable. Deciding whether two computations are behaviorally equivalent reduces to the halting problem by Rice's theorem, so full semantic convergence, sharing a name whenever two states behave the same, is unreachable by any algorithm. The honest statement is that structural convergence is constructible in principle with a tractability caveat, and behavioral convergence is impossible.

7.3 Userland phasing

The path to a real userland is mapped as a sequence of phases, stated here as the planned shape rather than as built work. U-0 is the smallest real userland, a single process with an explicit bounded capability space, five gated system calls, and an acquire-only bump allocator. U-1 follows. U-0 and U-1 are independent of the garbage collector and the SMP work. U-2 depends on the collector and U-3 depends on SMP, which is why those phases follow the substrates they need. At this writing U-0 is in progress and unverified. It is not yet Tier 1, and it is not presented as done. The currently loaded program is the static, embedded process of the Architecture, not the U-0 userland.

8 Limitations

CARMIX is roughly forty to forty-five percent of the way to a general-purpose operating system. The following are not built, and the list is the point of this section.

- **No real hardware.** Everything runs in QEMU. The bootable image runs the same path from a USB stick, but that has not been verified on a physical machine, and there is no real network transport, only ivshmem shared memory between two local instances.

- **Durable persistence is built but narrow.** A content-addressed object and a whole dematerialized process now survive a genuine cold reboot on a virtio-blk disk, re-verified by re-hash on every load (Tier 1). The scope is honest and narrow. There is a single durable writer, objects are at most one payload block, and the crash consistency is recovery-logic consistency under an ordered-prefix write-order model in QEMU, with real-hardware write reordering, cache and barrier semantics, and sub-sector tearing out of scope. There is still no general filesystem or namespace, the process image is embedded in the kernel, and the trust-store and nonce lifecycle state do not persist across a reboot.

- **No full allocator.** A process can grow its heap through an authority-bounded extend-heap backed demand-zero, and a userspace bump allocator runs on the granted pages, but the bump allocator has no free, there are no huge pages, and there is no production dedup-scan policy.

- **No dynamic linking.** The loaded program is a static executable.

- **Partial task-state dematerialization.** The content-addressed task object today is the registers and the used stack only. The page-table root and the capability slots stay resident and are not yet part of it.

- **PM2 was serviced through the page-fault core directly.** The idle-page rematerialization milestone calls the same C page-fault service routine the live vector runs, rather than triggering a live fault on a freshly-unmapped low virtual address, because a live touch there halts the boot on the single-stack handler. The serial line says so on the wire, and the live page-fault entry is demonstrated separately by F1, US0, and C0.

- **Recoverable nested faults are not supported.** A fault during fault handling fails closed rather than recovering. Making it recoverable needs a separate interrupt stack (IST/TSS) and a per-fault service context (F5).

- **The fairness control rests on lighter research.** It bounds future deficit growth and recovers the progress rate but does not refund the penalty already paid, and it accounts for one peer rather than many. It is a first measured control, not a proven-optimal fairness algorithm.
- **Deduplication leakage is partly closed, not eliminated.** Domain-partitioned deduplication closes the cross-domain store-timing channel at a measured storage cost (Tier 1), and the cross-domain leak-freedom rests on the argued impossibility bound (Tier 3). The within-domain timing channel remains and is accepted, because the observer is already in that domain. The bounded-residual mitigations, timing normalization and a randomized threshold, are not built, and a concurrent cross-core attacker is not exercised because it depends on the SMP work.
- **SMP is a foundation only.** A second CPU is up with per-CPU state, a working inter-processor interrupt path, and a cross-core test that shows genuine parallelism (Tier 1). The multi-core consistent capture and the DRCC result are not validated on it. The DRCC result is machine-checked in the minimal model of `proofs/CarmixDrcc.v` (Qed under `coqc` 8.20.1, under named model-to-machine assumptions) and first-exercise-measured on two real cores (U-3, CV8) over the exercised workloads and schedules only; its validation gap has narrowed now that the foundation exists, but CARMIX does not yet exhibit the multi-core cut (Tier 2).
- **Garbage collection and convergence have limits.** The durable collector is single-CPU. Concurrent tracing against a live multi-threaded mutator and cross-core reclamation are not built and wait on the SMP work. Content-addressed convergence is not built at all. Its structural canonical form is constructible only in principle, polynomial for bounded-degree graphs while CARMIX's arity is unmeasured, and its behavioral form is undecidable (Tier 3), a hard semantic-naming boundary rather than an engineering gap.
- **Userland is U-0 in progress, not a full userland.** The userland phasing is a research map. U-0, the smallest real userland, is in progress and unverified at this writing, and the later phases are planned, not built (Tier 3).
- **The PM0 stall is a known accepted failure.** A ring-3 readback in the per-process-memory stage stalls. It predates this work, is reported on the wire on every run rather than hidden, and is accepted, not fixed.
- **The trust-root bootstrap is open.** The destination's first authority public key is baked in. How a destination obtains and proves that first key, through a certificate authority, identity proofing, or an out-of-band channel, is the one irreducible bootstrap assumption and is not built. There is also no trustworthy clock for the expiry check, only a tick counter, and one authority key whose compromise is unrecoverable.
- **Input is polled PS/2.** Real modern hardware needs a USB HID stack over xHCI. The desktop is software-rendered and runs no third-party graphical applications.
- **The proof is the abstract algebra.** It is not a refinement of the C code. The runtime correspondence is demonstrated by the attack tables, not proven.

9 Related Work

Component and safe-language operating systems. Theseus structures the operating system as many small interchangeable cells with state ownership tracked by the language, to make live evolution and fault recovery tractable [11]. Twizzler builds an operating system around a single global data-centric address space over persistent memory [12]. Hubris is a small statically-defined memory-protected embedded operating system with no dynamic task creation [13]. Unikraft composes a specialized unikernel from modular library components [14]. CARMIX differs by making content-addressed rematerialization, not the module, the namespace, or the language, the unit through which it expresses context switch, fault, migration, crossing, load, and deschedule as one operation.

Capability and microkernel systems. seL4 is a microkernel with a complete machine-checked functional-correctness proof and a capability access-control model [7]. EROS and its ancestor KeyKOS are capability-based systems with transparent orthogonal persistence by periodic checkpoint [8, 9]. Genode is a capability-based component framework [10]. CARMIX shares the capability discipline but adds the anti-amplification re-mint as a migration and crossing primitive, ties it to a machine-checked authority property, and unifies it with content-addressed transfer. Its proof covers that authority algebra rather than full kernel functional correctness.

Migration and page deduplication. MITOSIS migrates serverless function state across machines efficiently [15], and content-addressed or hash-based virtual-machine and page migration has been built and measured. KSM merges identical anonymous pages in Linux by content scan [16]. The nearest active project

content-addresses program binaries, naming a function by the hash of its code and sharing it across a cluster. CARMIX content-addresses live computation state, the running memory graph, rather than code or post-hoc page scans, and re-mints the guarding capabilities during the transfer, which the page-merge and binary-addressing lines do not do.

Content-addressed stores and hashing. Git names objects by the SHA hash of their content [17], Nix builds a content-addressed package store [18], and IPFS is a content-addressed distributed file system [19]. These address files, packages, and blocks at rest. CARMIX applies the same naming to live execution state and to physical-memory residency. BLAKE3 is the hash function [20], chosen for speed. The store hashes only at commit and epoch boundaries, never in an execution loop.

Replacement and working-set theory. The residency manager and the scheduling policy rest on the classical working-set model of program locality [21]. CARMIX's contribution there is to anchor the dematerialize decision in a measured rematerialize cost and a live break-even, and to compose the hardware dirty bit with content-addressed dirty tracking, not a new replacement algorithm.

Durable content-addressed storage and reclamation. Venti is a durable content-addressed archival store whose blocks are named and verified by their hash [22], and CARMIX's persistence substrate borrows its self-verifying append-only model and its separation of storage from reclamation. Reference counting and the broader reclamation literature are standard [23]. CARMIX's contribution there is narrow, a durable refcount table reconstructed across a cold reboot with a tombstone-as-commit crash discipline, and acyclicity guaranteed by the content-addressed naming and machine-checked rather than assumed, so no cycle collector is needed.

Deduplication side channels. Cross-user deduplication leaks the existence of content through timing and occupancy [24], and the impossibility of cross-domain leak-freedom with cross-domain sharing is an established bound [25]. CARMIX applies that bound directly, partitioning deduplication by authority domain to close the cross-domain channel and paying the duplicate storage the bound makes unavoidable. The copy-on-write break channel does not apply, because CARMIX objects are immutable.

Canonicalization and consistent cuts. The convergence verdict rests on graph canonicalization and hash-consing [26] and on the bounded-valence graph isomorphism result that gives polynomial-time tractability only for bounded degree [27, 28]. The DRCC result rests on the release-consistency memory model [29], the data-race-free program class [30], the consistent-snapshot and happened-before line [31, 32], and Rice's theorem for the undecidability of behavioral equivalence [33]. CARMIX's parts here are derivations and a verdict, not standalone discoveries.

The unoccupied axis is the synthesis. No prior system makes content-addressed rematerialization under an authority-monotonic re-mint the single operation behind context switch, fault, migration, crossing, load, and deschedule, with the authority bound machine-checked.

10 Conclusion and Future Work

CARMIX is a research operating system organized around one principle, that a context switch, a page fault, a cross-machine migration, a user and kernel crossing, a program load, and a process deschedule are one operation: the rematerialization of content-addressed state under a re-minted anti-amplification capability ceiling, over BLAKE3 hashes. The contribution is the principle and its tested demonstration. The system boots, runs a capability-guarded WebAssembly executor, rematerializes state on one machine and across two, persists a content-addressed object and a whole process across a genuine cold reboot, reclaims unreachable objects with a durable collector, partitions deduplication by authority domain to close the cross-domain timing channel, brings up a second CPU as a multi-core foundation, enforces one anti-amplification gate at every boundary with the adversarial tables to show it, and machine-checks the core authority property in Coq under one stated collision-resistance hypothesis. Those are the Tier 1 results. It is not a finished operating system, and the Limitations section states what is missing.

Future work follows the open items, and the Tier 2 and Tier 3 results set the highest-value next steps. The DRCC result should be validated on the now-built SMP foundation by exhibiting a multi-core consistent-capture cut, which is the open multi-core question. Content-addressed convergence should be built, which is the high-ceiling open question, bounded above by the undecidability of behavioral equivalence. The userland phases should be completed past the in-progress U-0. The proof should be extended to a machine-checked refinement of the C backend against the abstract algebra, and a verified BLAKE3 should replace the vendored one once a verified BLAKE3 is confirmed to exist. The task-state object should grow to include the page-table root and the capability slots, recoverable nested faults need an interrupt stack and a per-fault service context, and the scheduling policy and the fairness control need a learned predictor and a multi-peer treatment. The platform needs real hardware bring-up, a USB HID stack, a general filesystem and namespace over the durable store, a full allocator, dynamic linking, and a real network transport. The trust root needs a bootstrap for

the first authority key, persistent rollback-resistant trust and nonce state, a trustworthy clock, and revocation at scale. Each of these is an honest open item rather than a hidden one, and the credibility of the demonstrated parts is meant to rest on that honesty.

References

[1] D. M. Ritchie and K. Thompson. The UNIX Time-Sharing System. *Communications of the ACM*, 17(7):365–375, 1974.

[2] R. Pike, D. Presotto, S. Dorward, B. Flandrena, K. Thompson, H. Trickey, and P. Winterbottom. Plan 9 from Bell Labs. *Computing Systems*, 8(3):221–254, 1995.

[3] R. Wahbe, S. Lucco, T. E. Anderson, and S. L. Graham. Efficient Software-Based Fault Isolation. In *Proceedings of the 14th ACM Symposium on Operating Systems Principles (SOSP)*, pages 203–216, 1993.

[4] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. F. Bastien. Bringing the Web up to Speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 185–200, 2017.

[5] D. J. Bernstein, T. Lange, and P. Schwabe. The Security Impact of a New Cryptographic Library. In *Progress in Cryptology – LATINCRYPT 2012*, LNCS 7533, pages 159–176, 2012.

[6] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang. High-Speed High-Security Signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012.

[7] G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, and S. Winwood. seL4: Formal Verification of an OS Kernel. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, pages 207–220, 2009.

[8] J. S. Shapiro, J. M. Smith, and D. J. Farber. EROS: A Fast Capability System. In *Proceedings of the 17th ACM Symposium on Operating Systems Principles (SOSP)*, pages 170–185, 1999.

[9] N. Hardy. KeyKOS Architecture. *ACM SIGOPS Operating Systems Review*, 19(4):8–25, 1985.

[10] N. Feske. *Genode Operating System Framework Foundations*. Genode Labs, 2015.

[11] K. Boos, N. Liyanage, R. Ijaz, and L. Zhong. Theseus: An Experiment in Operating System Structure and State Management. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 1–19, 2020.

[12] D. Bittman, P. Alvaro, P. Mehra, D. D. E. Long, and E. L. Miller. Twizzler: A Data-Centric OS for Non-Volatile Memory. In *Proceedings of the 2020 USENIX Annual Technical Conference (ATC)*, pages 65–80, 2020.

[13] Oxide Computer Company. Hubris: A Memory-Protected Microcontroller Operating System. Technical documentation, 2021.

[14] S. Kuenzer, V.-A. Bădoiu, H. Lefeuvre, S. Santhanam, A. Jung, G. Gain, C. Soldani, C. Lupu, S. Teodorescu, C. Răducanu, C. Banu, L. Mathy, R. Deaconescu, C. Raiciu, and F. Huici. Unikraft: Fast, Specialized Unikernels the Easy Way. In *Proceedings of the 16th European Conference on Computer Systems (EuroSys)*, pages 376–394, 2021.

[15] D. Du, Q. Liu, X. Jiang, Y. Xia, B. Zang, and H. Chen. Serverless Computing on Heterogeneous Computers with Fast Function Provisioning. *(MITOSIS / serverless function migration line of work)*, 2023.

[16] A. Arcangeli, I. Eidus, and C. Wright. Increasing Memory Density by Using KSM. In *Proceedings of the Linux Symposium*, pages 19–28, 2009.

[17] S. Chacon and B. Straub. *Pro Git*. Apress, 2nd edition, 2014.

[18] E. Dolstra, M. de Jonge, and E. Visser. Nix: A Safe and Policy-Free System for Software Deployment. In *Proceedings of the 18th USENIX Large Installation System Administration Conference (LISA)*, pages 79–92, 2004.

[19] J. Benet. IPFS – Content Addressed, Versioned, P2P File System. arXiv:1407.3561, 2014.

[20] J. O'Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O'Hearn. *BLAKE3: One Function, Fast Everywhere*. Specification, 2020.

[21] P. J. Denning. The Working Set Model for Program Behavior. *Communications of the ACM*, 11(5):323–333, 1968.

[22] S. Quinlan and S. Dorward. Venti: A New Approach to Archival Storage. In *Proceedings of the 1st USENIX Conference on File and Storage Technologies (FAST)*, pages 89–101, 2002.

[23] R. Jones, A. Hosking, and E. Moss. *The Garbage Collection Handbook: The Art of Automatic Memory Management*. Chapman and Hall/CRC, 2011.

[24] D. Harnik, B. Pinkas, and A. Shulman-Peleg. Side Channels in Cloud Services: Deduplication in Cloud Storage. *IEEE Security & Privacy*, 8(6):40–47, 2010.

[25] F. Armknecht, J.-M. Bohli, G. O. Karame, and F. Youssef. Transparent Data Deduplication in the Cloud. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 886–900, 2015.

[26] J.-C. Filliâtre and S. Conchon. Type-Safe Modular Hash-Consing. In *Proceedings of the 2006 ACM SIGPLAN Workshop on ML*, pages 12–19, 2006.

[27] E. M. Luks. Isomorphism of Graphs of Bounded Valence Can Be Tested in Polynomial Time. *Journal of Computer and System Sciences*, 25(1):42–65, 1982.

[28] L. Babai. Graph Isomorphism in Quasipolynomial Time. In *Proceedings of the 48th Annual ACM Symposium on Theory of Computing (STOC)*, pages 684–697, 2016.

[29] K. Gharachorloo, D. Lenoski, J. Laudon, P. Gibbons, A. Gupta, and J. Hennessy. Memory Consistency and Event Ordering in Scalable Shared-Memory Multiprocessors. In *Proceedings of the 17th Annual International Symposium on Computer Architecture (ISCA)*, pages 15–26, 1990.

[30] S. V. Adve and M. D. Hill. Weak Ordering: A New Definition. In *Proceedings of the 17th Annual International Symposium on Computer Architecture (ISCA)*, pages 2–14, 1990.

[31] K. M. Chandy and L. Lamport. Distributed Snapshots: Determining Global States of Distributed Systems. *ACM Transactions on Computer Systems*, 3(1):63–75, 1985.

[32] L. Lamport. Time, Clocks, and the Ordering of Events in a Distributed System. *Communications of the ACM*, 21(7):558–565, 1978.

[33] H. G. Rice. Classes of Recursively Enumerable Sets and Their Decision Problems. *Transactions of the American Mathematical Society*, 74(2):358–366, 1953.